

M2S2 | Model & Methodology

Lifecycle Model

defines a set of processes, actions, and tasks and the order in which they are performed

Software Development Methodology

is a step-by-step guideline. It prescribes how engineers go about their work in order to move the system through its life cycle

Software lifecycle models

Build-and-Fix

incomplete LC software, small projects

Waterfall

feedback from the early stages of LC software

Rapid prototyping

quick prototype, early validation, requirement clarification

Incremental

delivered in increments, staged functionality growth

Spiral

multiple iterations with risk analysis

Comparative analysis

Model LC Advantages Disadvantages	Advantages	Disadvantages
Build-and-Fix	It is good for small, maintenance-free projects.	It is absolutely unsuitable for non-trivial projects
Waterfall	Clear project discipline, document-driven	The software may not meet the customer's requirements
Spiral	Combines the characteristics of all the models listed above	Suitable only for large internal projects; developers must be proficient in risk management
Incremental	The earliest possible return on investment; promotes maintainability	Requires an open architecture; may degenerate into a Build-and-fix
Rapid prototyping	Ensures compliance ACCORDING to customer requirements	It's tempting to reuse code that should be reimplemented

Software lifecycle methodologies

RUP

iterative, use-case driven, architecture-centric

Scrum

short sprints, early feedback, incremental delivery

XP

pair programming, TDD, frequent releases

MSS

ready-made solutions, structured enterprise framework

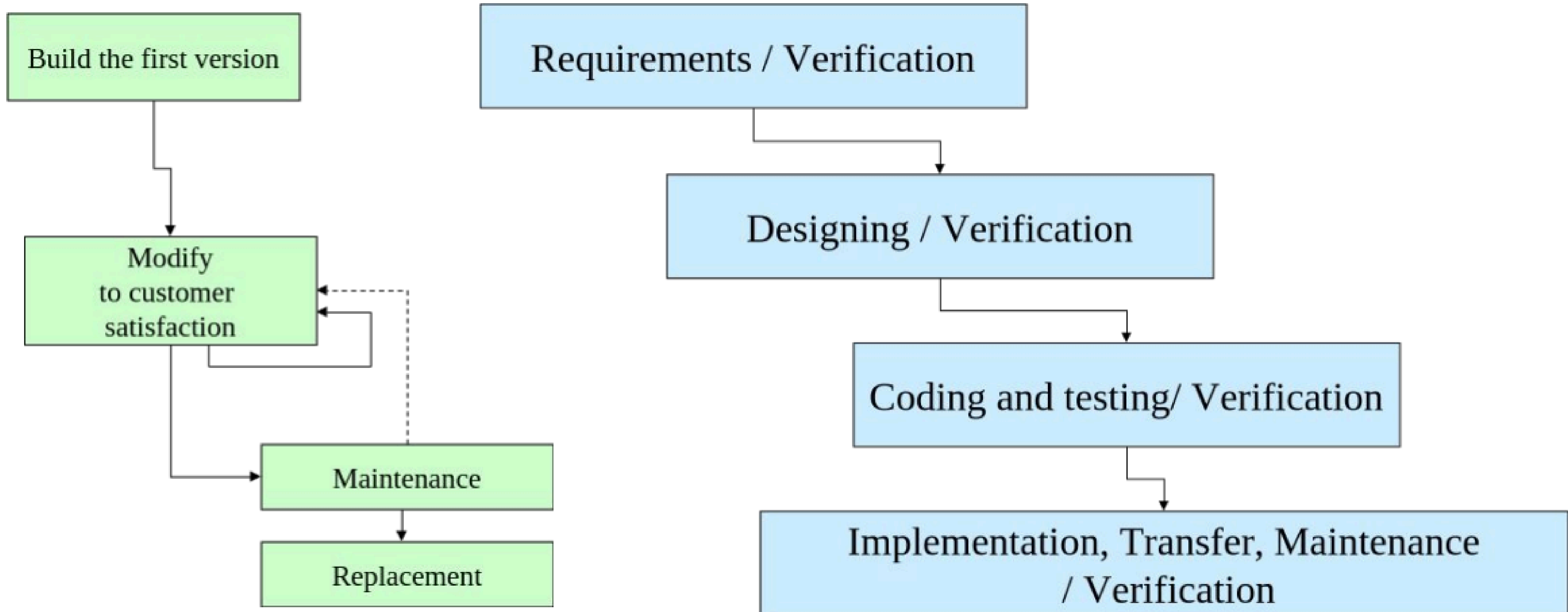
TSP

self-directed teams, metrics-driven, risk management

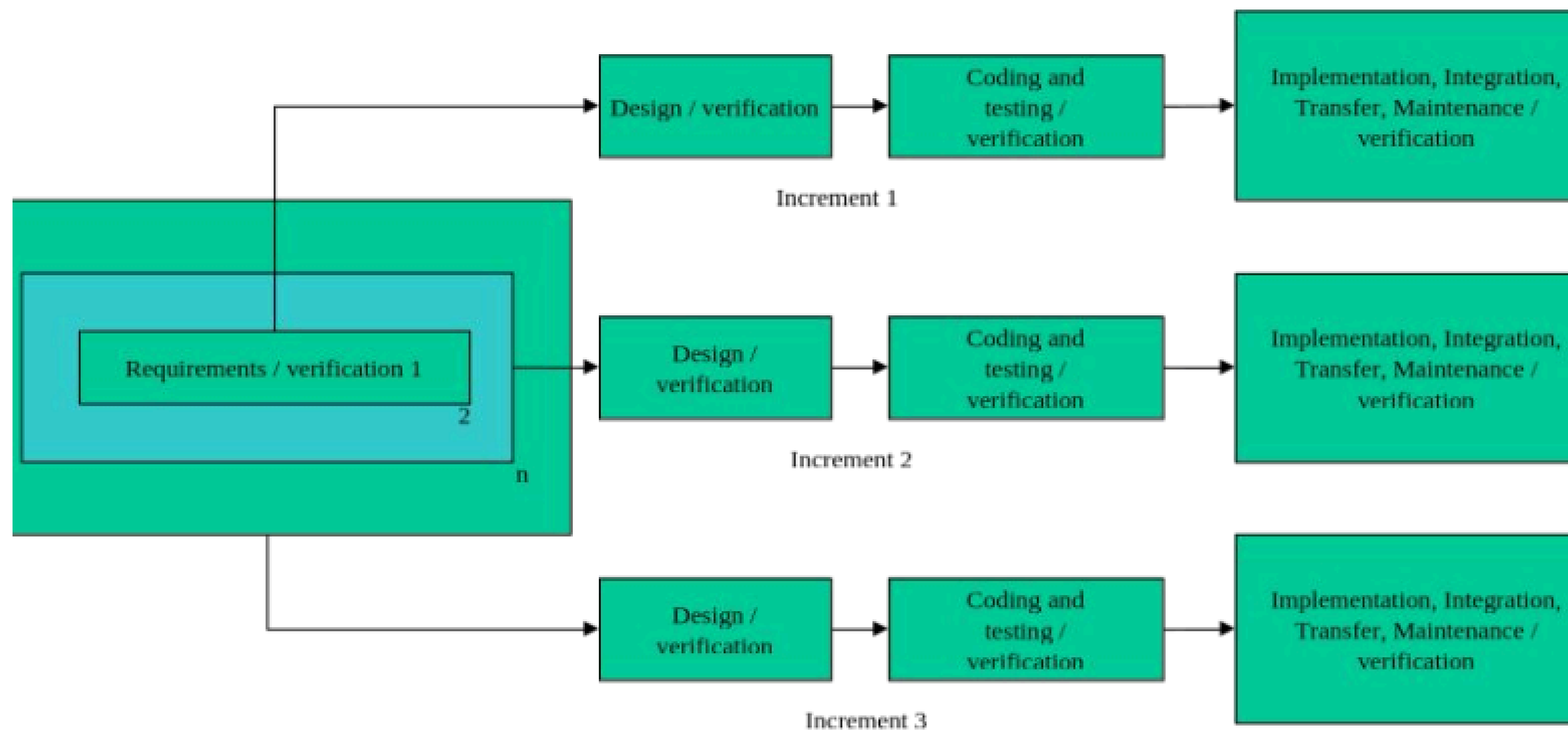
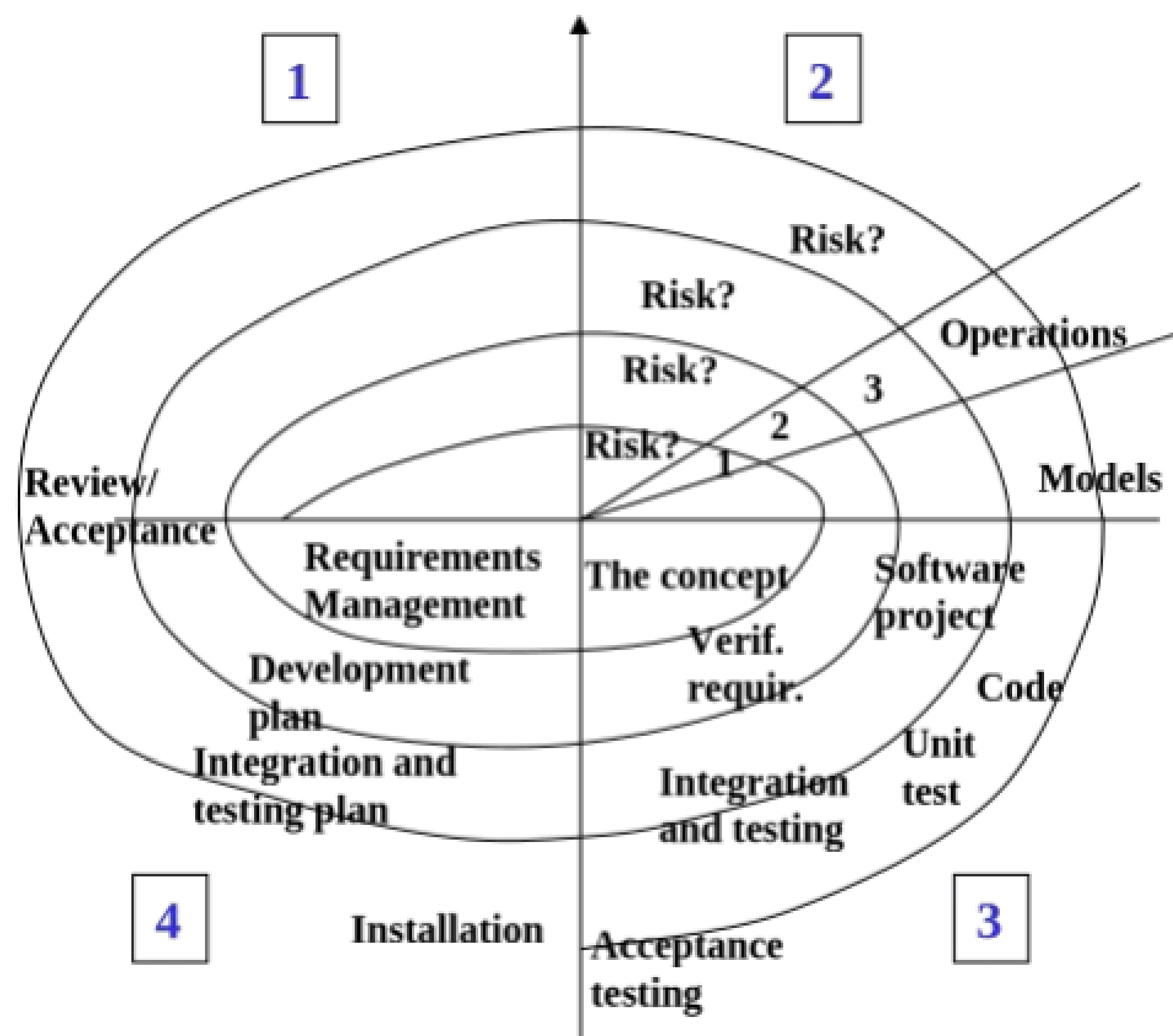
Comparative analysis

Methodology	Advantages	Disadvantages
RUP	well-structured, adaptable, supports risk management early, scalable for large projects	heavyweight, requires training, may be over-formalized for small teams
Scrum	simple artifacts, strong client involvement, transparency, adaptive planning	minimal documentation, weak outside project management, requires high discipline
XP	strong code quality, fast feedback, customer-driven, lightweight	requires customer on-site, high dependency on communication, weak documentation, some devs dislike pair programming

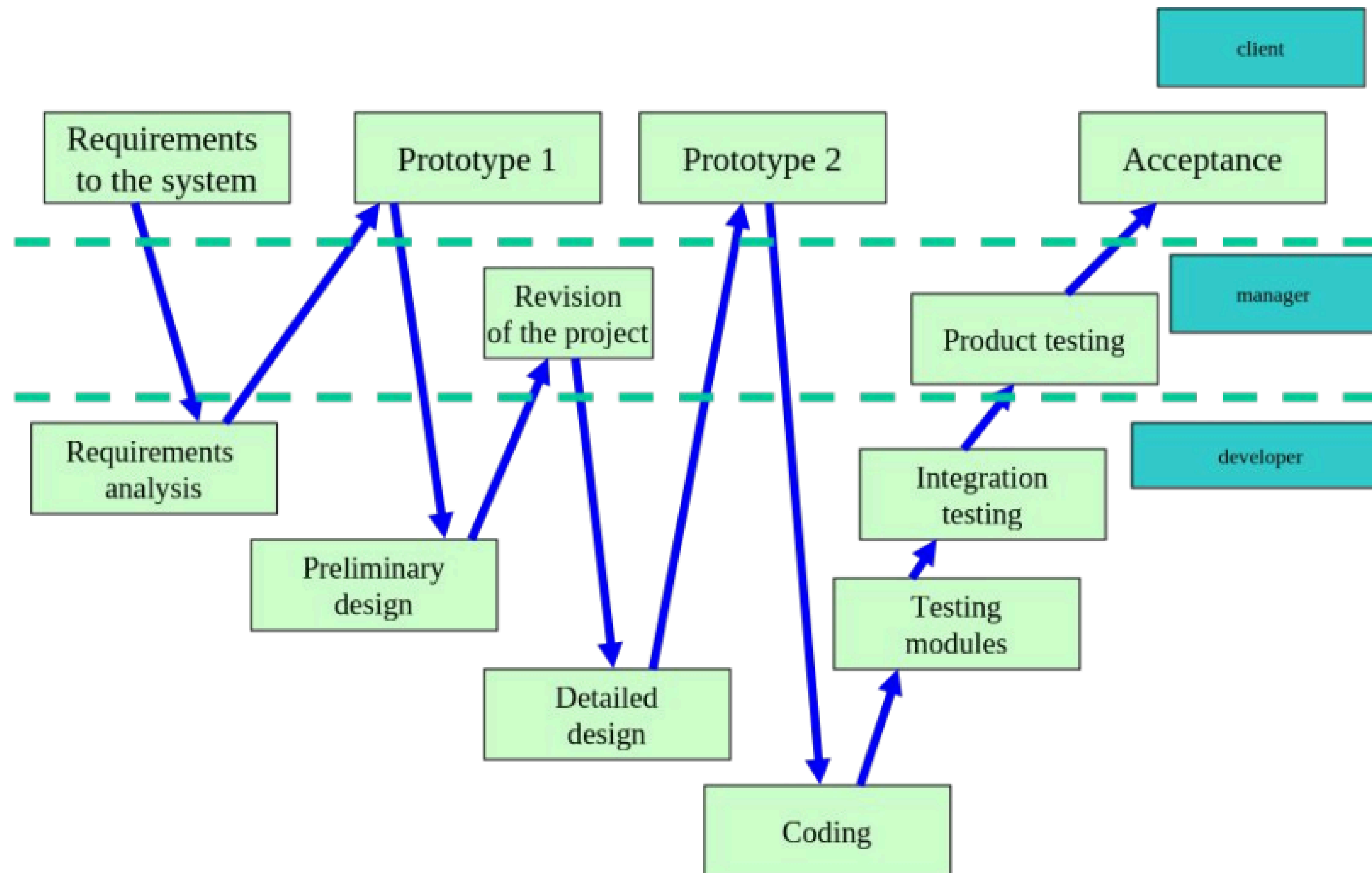
practice



Guess the Model



Guess the Model



Guess the Model